

RIME HACKATHON · DEMO PACKAGE

Continuum: Demo Day

What the demo website must show, second by second — including the voice barge-in ("just pick up the call") test.

Companion to: Continuum_Rime_Hackathon_Blueprint.pdf

Covers: Dashboard panel spec, minute-by-minute recording script, the new barge-in interrupt test, pre-demo setup, and the live evidence panel judges will see on screen

Target run time: 5:00

Contents

- 01 Why the Website Matters for Judging
- 02 Required Screens & Panels
- 03 The Barge-In / Voice Interrupt Test (New)
- 04 Minute-by-Minute Demo Script
- 05 On-Screen Event Log Schema
- 06 Pre-Demo Setup Checklist
- 07 Recording-Day Checklist
- 08 Evidence Panel → RIME_EVIDENCE.md Mapping

01 · Why the Website Matters for Judging

The demo website is not a decoration — it is how judges verify the hard voice problem was actually solved, without needing to read your code. Every claim in your README and RIME_EVIDENCE.md ("recap starts before the caller notices," "the whisper never leaks," "the interrupt stops audio in under 300ms") needs a visible, on-screen moment that proves it as it happens.

Treat the dashboard as a **live instrument panel**, not a marketing page. Its job during the 5-minute recording is to make invisible things visible: which audio track is playing, exactly when an event fired, and how many milliseconds elapsed between a trigger and a response. If a judge can't see the timestamp, they have to take your word for it — and "evidence and reproducibility" is 20% of the score.

Three things the site must always make obvious

- **Which audio track is active** — caller-facing vs. private whisper — at every moment, with a clear visual distinction (not just a log line).
- **Provider transparency** — Rime's model, voice, and endpoint visibly labeled, so it's unambiguous that Rime is carrying the core experience, not a placeholder.
- **Timestamps on every system event** — disconnect detected, reconnect matched, recap started, interrupt detected, recap halted, call answered — each with a millisecond-precision time, so latency claims are self-evidently backed by the same screen the judge is watching.

02 · Required Screens & Panels

Four screens/panels, all reachable without navigating away mid-recording (tabs or a single scrolling dashboard both work).

A — Header bar (always visible)

CONTINUUM

Thread: Z · Contact

Provider

Powered by Rime · model: Mist v3 · voice: "Marsh" · endpoint: us-east · transport: LiveKit Agents

Call state

RINGING — matched to paused thread

This bar must be on screen for the entire recording. It is the single fastest way for a judge to confirm eligibility (Rime is the active, primary provider) without scrubbing through the video.

B — Dual audio track visualizer

Caller-facing track (what Z hears)

|||||

ringback tone only

Private whisper track (what only the user hears)

|||||

Rime recap playing

Two separate, simultaneously-animated waveform lanes, clearly labeled. This is the single visual that proves the "never leaks to the caller" architectural invariant — when the whisper lane is active and the caller-facing lane shows nothing but ring tone, the fencing claim is self-evident on screen.

C — Live transcript & recap panel

Thread memory (yesterday)

Z asked about Q3 pricing · quoted \$400 · you said you'd confirm with finance

Recap being spoken now

"Z wanted the Q3 number, you'd said you'd check with finance."

Freshness flag: price is now \$420, was \$400

Captions of the recap text should appear in sync with the audio (word-by-word or sentence-by-sentence), so judges watching without sound still see the claim.

D — Event log (timestamped, append-only, always visible in a side panel)

Event log

14:02:11.204 call.disconnect_detected thread=Z
14:02:11.980 thread.marked_interrupted
09:14:02.001 call.inbound_ring caller=Z
09:14:02.040 thread.match_found confidence=1.00
09:14:02.310 recap.generation_started
09:14:03.512 recap.tts_first_audio Δ = 1.20s from match
09:14:05.880 barge_in.detected phrase="just pick up the call"
09:14:06.070 tts.playback_halted Δ = 190ms from barge-in
09:14:06.090 call.auto_answer_triggered

This log is arguably the most important panel in the whole site — it's what turns your verbal claims into machine-generated, timestamped proof, live, during the recording.

03 · The Barge-In / Voice Interrupt Test NEW

This is a genuinely good addition — it's a second hard voice problem stacked on top of the first one, and it demos extremely well because it's a natural, single-sentence human moment: the user says *"I know, just pick up the call"* while Rime is mid-recap, and the system must stop talking and act on the instruction almost instantly.

What has to be built for this to be real (not scripted)

- **Always-on VAD/ASR on the user's mic**, running continuously even while Rime's TTS is playing on the private track — this is full-duplex listening, not "wait for TTS to finish, then listen."
- **Immediate playback cancellation** the moment user speech is detected — cancel the in-flight Rime audio stream, don't let it fade out or finish the sentence.
- **Intent parsing on the barge-in utterance**, not just "stop." "Just pick up the call" is a command, not a generic interruption — it should trigger auto-answer. A plain "hold on" or "wait" should just pause/stop without auto-answering. Distinguish these two cases explicitly in your intent classifier.

New module to add to your architecture

Module	Responsibility	Key tech
Barge-In Handler	Runs VAD continuously on the user's private-track mic input during TTS playback. On detected speech: (1) immediately cancel/stop the Rime audio stream, (2) run fast intent classification on the captured utterance, (3) route to the matching action — auto-answer, resume recap, or stop entirely.	Streaming VAD (e.g. Silero/webrtcvad) + LiveKit's interrupt/cancel API on the TTS stream + a small intent classifier (few-shot prompt on a fast model, or a keyword/embedding match for hackathon speed)

New acceptance test

Test	Procedure	Pass criteria
Barge-in latency	While recap is playing, speak "I know, just pick up the call" and measure time from speech onset to TTS audio fully silent.	Under ~300ms; log both the <code>barge_in.detected</code> and <code>tts.playback_halted</code> timestamps and show the delta on screen.
Intent correctness	Run two utterances: an explicit action ("just pick up the call") vs. a generic interrupt ("hold on") and show the system responds differently to each.	Action phrase triggers auto-answer; generic phrase only halts audio without answering.
No orphaned audio	Inspect the caller-facing track during and after the barge-in.	Confirms the interrupt affected only the private track and produced no audio artifact on the caller's side (the caller was never able to hear the recap anyway, but this test proves the cancellation is clean, not just muted late).

This test pairs naturally with the "interruptibility" acceptance test already in the main blueprint (Section 12/13) — treat the barge-in test as the sharper, more demo-friendly version of that same requirement, and lead with it.

04 · Minute-by-Minute Demo Script

Total run time: 5:00. Keep the header bar and event log panel visible throughout — never cut away from them.

Time	On screen	Audio / spoken	Why it's here
0:00-0:15	Title card / dashboard header bar with Rime provider badge visible	Presenter: one-sentence problem statement	Immediately shows Rime is the primary provider (eligibility)
0:15-0:45	Thread list → open thread with Z; show yesterday's structured summary (panel C)	Presenter explains: "yesterday I was on a call with Z, quoted \$400, then got cut off"	Establishes the "before" state judges need to compare against
0:45-1:15	Live call in progress; transcript scrolling in panel C; then a forced disconnect — event log fires <code>call.disconnect_detected</code>	Live audio of the call, cut off mid-sentence	Shows the interruption actually happening, not simulated after the fact
1:15-1:45	Thread marked "interrupted"; cut to next day (timestamp jump shown on screen); Z's number rings in; event log fires <code>call.inbound_ring</code> + <code>thread.match_found</code>	Ringtone only, nothing else	Sets up the pre-answer window — the core hard problem
1:45-2:15	Dual waveform panel (B) active: whisper lane lights up, caller-facing lane stays flat/ring-only; captions of recap text appear in panel C; freshness flag turns red (\$400 → \$420)	Rime speaking the recap on the private track only (audible to viewer via a picture-in-picture "user's ear" audio feed)	Proves pre-answer whisper + fencing + freshness catch simultaneously
2:15-2:35	Event log fires <code>barge_in.detected</code> then <code>tts.playback_halted</code> with the millisecond delta on screen; whisper waveform cuts instantly	Presenter (as the user) says: "I know, just pick up the call"	The new barge-in test — shows real full-duplex interrupt handling
2:35-2:45	Event log fires <code>call.auto_answer_triggered</code> ; call state flips to CONNECTED	Call connects; Z says "hey, sorry, we got cut off"	Proves the barge-in phrase was parsed as an action, not just a stop
2:45-3:10	Split-screen or sequential playback of exactly what Z heard from the moment of ring to answer	Normal ring → normal "hello" — no recap audio present at all	Direct proof of zero added latency/experience impact for the caller
3:10-3:40	Second, shorter scenario: instant-connect call with no ring buffer; user silently invokes the assistant mid-call; whisper lane lights up briefly	User (quietly): "remind me what he wanted" → whispered answer	Shows the fallback path works too, not just the ideal-case ring window
3:40-4:15	Evidence panel (Section 08 of this doc): latency numbers, pass/fail checklist, all green	Presenter reads out the two headline numbers: recap start latency, barge-in response latency	Directly serves "evidence and reproducibility" scoring
4:15-4:45	One-slide architecture recap (from the main blueprint, Section 06 diagram)	Presenter: 2-sentence tech stack summary — LiveKit, Rime model/voice, the fencing invariant	Judges' quick mental map of what they just watched
4:45-5:00	Repo link / team names on screen	Close	Standard wrap

05 · On-Screen Event Log Schema

Emit every event below as a real, timestamped record from your backend to the dashboard over a websocket (not hand-typed captions) — this is what makes the log credible as evidence rather than decoration.

```
{
  "ts": "2026-09-05T09:14:03.512Z",
  "event": "recap.tts_first_audio",
  "thread_id": "z-contact-01",
  "meta": {
    "since_event": "thread.match_found",
    "delta_ms": 1201
  }
}
```

Minimum required event types

Event	Fired when
call.disconnect_detected	An abrupt call end is identified (no closing turn)
thread.marked_interrupted	The paused thread is persisted with its last state
call.inbound_ring	A new inbound call starts ringing
thread.match_found	Caller ID matched against a stored paused thread
recap.generation_started / recap.text_ready	LLM recap generation begins / completes
freshness.discrepancy_found	A checked fact differs from what was last discussed
recap.tts_first_audio	First audible sample of the whispered recap plays
barge_in.detected	VAD detects user speech while TTS is playing
tts.playback_halted	TTS audio stream is fully stopped
barge_in.intent_classified	The captured utterance is classified (action vs. generic stop)
call.auto_answer_triggered	The system answers the call on the user's behalf

06 · Pre-Demo Setup Checklist

- ☐ Seed the Thread Memory Store with yesterday's Z conversation, including the \$400 quote, so the "next day" scenario has real prior state, not an empty thread.
- ☐ Set the mock pricing API's current value to \$420 (different from the seeded \$400) so the freshness flag fires deterministically.
- ☐ Confirm the header bar's provider badge reflects the actual model/voice/endpoint you'll use in the recording — don't leave a stale placeholder value.
- ☐ Dry-run the barge-in phrase 3–5 times beforehand and confirm the intent classifier reliably distinguishes "just pick up the call" (action) from a generic "wait"/"hold on" (stop-only), since this is the riskiest live moment in the script.
- ☐ Verify the websocket event feed reconnects cleanly if the dashboard tab is refreshed mid-recording — you do not want a blank event log during the take.
- ☐ Record a clean isolated clip of exactly what the caller-side (Z) audio sounds like, for the 2:45–3:10 segment, independent of the live take, as a fallback in case live capture is noisy.
- ☐ Reset all timestamps/thread state to a known clean baseline immediately before hitting record.

07 · Recording-Day Checklist

- ☐ Screen-record the full browser window including the header bar and event log — never crop them out for a "cleaner" shot.
- ☐ Capture two audio sources separately if possible: the caller-facing feed and the user's private whisper feed, so you can prove separation in post if needed.
- ☐ Do at least 2 full takes; the barge-in timing (2:15–2:35) is the single most failure-prone moment — budget time to redo just that segment if the VAD misfires.
- ☐ After recording, scrub the video against the event log timestamps once to confirm every spoken claim in the voiceover lines up with a visible, timestamped event on screen.
- ☐ Export the final event log for the full demo run and attach it as a raw artifact alongside the video — this becomes part of your reproducibility evidence.

08 · Evidence Panel → RIME_EVIDENCE.md Mapping

The on-screen evidence panel shown at 3:40–4:15 should mirror your written RIME_EVIDENCE.md exactly, so judges can cross-check the video against the repo in seconds.

On-screen metric	Source event(s)	Where it lives in RIME_EVIDENCE.md
Recap start latency (ring → first audio)	<code>thread.match_found</code> → <code>recap.tts_first_audio</code>	"Pre-answer latency" test result
Barge-in response latency	<code>barge_in.detected</code> → <code>tts.playback_halted</code>	"Barge-in latency" test result
Zero caller-side impact	Isolated caller-facing track recording	"Zero impact on caller" test result
Freshness catch	<code>freshness.discrepancy_found</code>	"Freshness catch" test result
Context fencing	Dual-track visualizer screenshot, both lanes simultaneously	"Context fencing" test result
Intent correctness	<code>barge_in.intent_classified</code> for both test phrasings	"Intent correctness" test result (new, Section 03)

This document is a companion to Continuum_Rime_Hackathon_Blueprint.pdf. Build the dashboard panels in Section 02 first, wire the event schema in Section 05 second, then rehearse the script in Section 04 — in that order, the demo becomes mostly a matter of pressing record.